PATCHCOURT

Multi-Agent Evidence-Tiered AI Code Review System

MASTER REFERENCE SHEET

Stack	Python 3.12 · FastAPI · LangGraph · Next.js 15
LLM Support	Mock · OpenAI · Gemini · Claude · Ollama
Tools	Semgrep · Bandit · Gitleaks · Radon · SonarQube
Infra	Docker Compose · PostgreSQL · Redis · Qdrant
Verdict	MERGE | NEEDS_REVIEW | BLOCK
Hard Rule	BLOCK never rests on T5 (LLM-only) evidence alone

Covers: Architecture · Pipeline · Evidence Tiers · Agents · Debate · Judge · Adjudication · QA-Checker · Taint Inference · SonarQube Baseline · Dashboard · API · Docker · Deployment · Troubleshooting · Research Basis

1. PROJECT OVERVIEW

PatchCourt is a fully containerised, multi-agent AI system that reviews GitHub Pull Requests using **evidence-tiered reasoning**. Three specialist LLM agents (Security, Quality, Pragmatist) analyse a PR in parallel. Every claim is anchored to a trust tier (T1–T5) derived from static-analysis tools, repository policies, git precedent, or LLM reasoning. Disagreements trigger a bounded, retrieval-gated debate. A deterministic judge produces a final score and one of three verdicts.

1.1 Core Design Principles

Evidence-first	Every claim must cite a tier. LLM reasoning alone (T5) can never force a BLOCK.
Deterministic judge	Scoring is pure arithmetic — no LLM in the verdict path.
Sandboxed tools	PR code runs inside an isolated Docker container (--network none).
Offline-capable	Full pipeline runs with zero credentials using mock LLM + synthetic scanner.
Pluggable LLM	Switch provider at runtime via API — no container restart needed.
Audit trail	Every review is persisted to PostgreSQL with full claim/evidence detail.

1.2 Verdict System

Verdict	Score Threshold	Meaning	Colour
MERGE	< 4.0	No blocking issues found	Green
NEEDS_REVIEW	4.0 – 9.99	Issues present; human review required	Amber
BLOCK	≥ 10.0	Critical issues; merge must be blocked	Red

■ *BLOCK is downgraded to NEEDS_REVIEW if the score crosses the threshold only via T5-only claims. This is enforced in judge/scoring.py and unit-tested.*

2. ARCHITECTURE

2.1 Five-Layer Stack

L1 · Presentation	CLI (python -m patchcourt) · FastAPI REST · Next.js 15 Dashboard
L2 · Async & Persistence	Celery workers · Redis broker · PostgreSQL audit trail
L3 · Intelligence	Security / Quality / Pragmatist agents · Conflict detector · Debate · Judge
L4 · Evidence	Evidence engine · Docker sandbox · Semgrep/Bandit/Gitleaks/Radon · Qdrant RAG
L5 · Data & External	GitHub REST API · LLM provider · .env config

2.2 LangGraph Pipeline (graph.py)

The review pipeline is wired as a LangGraph StateGraph. Each node is an async function that reads from and writes to a shared GraphState TypedDict. Nodes run sequentially except the three agent nodes which run in parallel after adjudication.

Node	Input State Keys	Output State Keys	Notes
`ingest`	pr (empty)	pr (populated)	Skipped in demo/offline mode if pr already has file_contents
`rag`	pr	rag_results	Clears Qdrant, indexes PR files, queries top-8 chunks
`evidence`	pr, rag_results	tool_claims, raw_findings	Runs sandbox or local tools; normalizes findings to Claims
`adjudicate`	pr, raw_findings	correlations, finding_assessments	FindingValidator + taint inference (IRIS-inspired)
`security`	pr, rag_results, tool_claims	security_claims	SecurityAgent LLM call
`quality`	pr, rag_results, tool_claims	quality_claims	QualityAgent LLM call
`pragmatist`	pr, rag_results, tool_claims	pragmatist_claims	PragmatistAgent LLM call
`merge_all`	*_claims	all_claims	Merges tool + LLM claims; runs corroborate_claims()
`detect`	all_claims	conflicts	detect_conflicts() — routes to debate or judge
`debate`	conflicts, pr, finding_assessments	debated_claims, finding_assessments	≤2 rounds; QA-Checker runs after; mutates qa_verdict/qa_note
`judge`	all_claims, debated_claims, finding_assessments	report (ReviewReport)	Deterministic scoring; safe_verdict() enforces T5 rule

2.3 Module Layout

`patchcourt/agents/`	schemas.py (all Pydantic models) · base.py · security.py · quality.py · pragmatist.py
`patchcourt/ingest/`	github.py — fetch PR metadata, diff, raw file contents, base file contents
`patchcourt/rag/`	store.py — Qdrant chunking, indexing, query (real embeddings or hash fallback)

`patchcourt/evidence.py`	EvidenceEngine, normalize_findings(), corroborate_claims()
`patchcourt/tools/`	base.py · registry.py · semgrep/bandit/gitleaks/radon/synthetic adapters
`patchcourt/sandbox/`	runner.py — docker run --rm --network none per-scan container
`patchcourt/adjudication/`	validator.py · contextual.py · correlate.py · rules.py
`patchcourt/debate/`	conflict.py · debate.py — bounded retrieval-gated debate
`patchcourt/judge/`	scoring.py — claim_score(), safe_verdict(), build_report()
`patchcourt/baseline/`	sonarqube_baseline.py — fetch issues, on-demand scan, compare_findings()
`patchcourt/db/`	models.py · session.py · audit.py — SQLAlchemy PostgreSQL audit
`patchcourt/worker/`	tasks.py — Celery review_pr task
`patchcourt/api/`	app.py — all FastAPI endpoints · webhook.py
`patchcourt/qa_checker.py`	QA-Checker agent — validates debate conclusions (CodeAgent-inspired)
`patchcourt/taint.py`	IRIS-inspired taint inference — LLM-confirmed source/sink flows
`patchcourt/llm.py`	Provider dispatch: mock / openai / claude / gemini / ollama
`patchcourt/config.py`	Settings (pydantic) + TIER_WEIGHTS dict
`patchcourt/graph.py`	LangGraph StateGraph wiring — build_graph(), run_review()
`dashboard/src/`	Next.js 15 app — app/ · components/ · lib/

3. EVIDENCE TIERS & SCORING

3.1 Evidence Tier Weights (config.py)

Tier	Weight	Source	Can Block Alone?
T1	1.0	Direct static-analysis tool flag with file:line	YES
T2	0.8	Two independent tools agree on the same file:line (corroborated)	YES
T3	0.5	Retrieved from repository policy / coding standards (RAG)	NO
T4	0.4	Git precedent — pattern from history / similar PRs (RAG)	NO
T5	0.1	LLM reasoning only — no tool or policy backing	NEVER

3.2 Scoring Formula (judge/scoring.py)

Each claim is scored independently then summed per file, then across all files:

```
score = severity × tier_weight × corroboration_factor × confidence × fault_multiplier
```

Parameter	Range	Description
`severity`	1–5	1=info, 2=low, 3=medium, 4=high, 5=critical
`tier_weight`	0.1–1.0	From TIER_WEIGHTS dict (T1=1.0 … T5=0.1)
`corroboration_factor`	1.0–1.75	1.0 + 0.25 × (agents agreeing on same file). Max 1.75 with 3 agents.
`confidence`	0.0–1.0	Agent-reported confidence in the claim
`fault_multiplier`	0.85–1.15	±15% nudge for T1/T2 tool claims only. High-fault rules (SQL, shell, secrets) → 1.15; cosmetic → 0.85

3.3 Hard BLOCK Constraint

safe_verdict() in scoring.py computes two scores: overall_score (all claims) and non_t5_score (claims with at least one T1–T4 evidence item). If overall_score ≥ 10.0 but non_t5_score < 10.0, the verdict is downgraded from BLOCK to NEEDS_REVIEW. This is unit-tested in tests/test_judge.py.

3.4 Adjudication Statuses (validator.py)

Status	Meaning	Typical Cause
SUPPORTED	Unsafe pattern independently reproduced on the flagged line	danger pattern matched AND no safe pattern present
CONTRADICTED	Code already uses a safe pattern, or scanner match not reproducible	safe pattern found on line, or rule not reproducible
UNCERTAIN	Evidence insufficient; escalate for human review	Rule uncatalogued, or corroborated but not reproduced
OUT_OF_SCOPE	Finding is outside the reviewed change	File not in PR's changed_files set

3.5 Three Specialist Agents

Agent	Focus	Typical Claims
SecurityAgent	Vulnerabilities, secrets, injection, unsafe deserialization	SQL injection, hardcoded credentials, pickle.load(), shell=True
QualityAgent	Code correctness, maintainability, complexity, dead code	Unused variables, missing args, high cyclomatic complexity
PragmatistAgent	Practical risk, deployment impact, real-world exploitability	Risk-weighted severity, context-aware blocking recommendations

4. RESEARCH-BACKED FEATURES

Five features were implemented from analysis of 8 research papers. Each is traceable to a specific paper and implemented in a specific module.

Feature	Paper Inspiration	Module	What It Does
QA-Checker Agent	CodeAgent — autonomous communicative agents for code review	`patchcourt/qa_checker.py`	Validates debate conclusions: checks T5-only BLOCK, vague summaries, empty arguments, evidence-tier inconsistencies. Mutates qa_verdict/qa_note.
FP/FN Root Cause Tagging	Empirical Study of False Negatives and Positives of Static Code Analyzers	`adjudication/validator.py`	7-category taxonomy (Flawed Rule Spec, Missing Cases, Unhandled Language Features, etc.) tagged on every CONTRADICTED/UNCERTAIN assessment.
Fault-Risk Weighting	Some SonarQube Issues have a Significant but Small Effect on Faults	`judge/scoring.py`	±15% nudge (_fault_multiplier) on T1/T2 tool claims only. High-fault rules (SQL, shell, secrets) → 1.15×; cosmetic → 0.85×.
LLM Taint Inference	IRIS — LLM-assisted taint specification inference	`patchcourt/taint.py`	4 taint specs (SQL/CMD/PATH/XSS). Pattern-based source/sink extraction + LLM confirmation. Returns FindingAssessment with tool='taint-inference'.
PR Diff-Only Scan	Bugdar — bug detection focused on changed code	`api/app.py POST /api/baseline/diff`	Scans only changed files/lines. Filters issues to in_pr=True. Faster than full scan for large repos.

4.1 QA-Checker Detail (qa_checker.py)

`_check_t5_block`	INCONSISTENT if debate concludes BLOCK but all evidence is T5. Patches verdict_summary.
`_check_evidence_tier_consistency`	INCONSISTENT if severity ≥ 4 claim has only T5 evidence.
`_check_rationale_vagueness`	NEEDS_REWORK if verdict_summary is empty or < 12 words of vague language.
`_check_supporting_opposing_balance`	NEEDS_REWORK if either prosecution or defense argument is empty.
`qa_check_assessment()`	Validates FindingAssessment: SUPPORTED needs T1/T2; CONTRADICTED needs rationale; block action only on SUPPORTED.

4.2 Taint Inference Detail (taint.py)

Spec ID	CWE	Source Patterns	Sink Patterns
`TAINT-SQL`	CWE-89	request.args/form/json, input(), sys.argv, os.environ	execute/query/cursor(... +, f-string SELECT, string + SELECT
`TAINT-CMD`	CWE-78	request.*, input(), sys.argv	subprocess.call/run/Popen, os.system/popen, eval()
`TAINT-PATH`	CWE-22	request.*, input()	open(), os.path.join/exists, pathlib.Path
`TAINT-XSS`	CWE-79	request.*	render_template_string, Markup(), innerHTML=, document.write()

■ *Flows are only flagged if source and sink are within 50 lines (same-function heuristic). LLM confirmation is skipped in mock mode; pattern-based detection is used instead. Confidence threshold: 0.4 minimum to emit a FindingAssessment.*

5. DEPLOYMENT

5.1 Container Inventory

Container	Image	Port	Role
`patchcourt-api-1`	patchcourt-api:latest (python:3.12-slim)	8000	FastAPI backend, LangGraph pipeline, sandbox orchestration
`patchcourt-dashboard-1`	patchcourt-dashboard:latest (node:22-alpine)	3001→3000	Next.js 15 frontend dashboard
`patchcourt-postgres-1`	postgres:16-alpine	5432 (internal)	Audit log persistence (SQLAlchemy)
`patchcourt-redis-1`	redis:7-alpine	6379 (internal)	Celery task broker
`patchcourt-qdrant-1`	qdrant/qdrant:latest	16333→6333	RAG vector store for policy/precedent retrieval
`patchcourt-worker-1`	patchcourt-api:latest	— (profile: worker)	Celery async worker for webhook-triggered reviews
`patchcourt-sonar-sonarqube-1`	sonarqube:community	9000	Static analysis baseline; embedded H2 DB
`patchcourt-sandbox (build-only)`	patchcourt-sandbox:latest	— (profile: tools)	Semgrep+Bandit+Gitleaks+Radon. Spawned per-scan via docker run --rm

5.2 Docker Deployment Commands

Action	Command
Build all images	`docker compose build && docker compose build sandbox`
Start core stack	`docker compose up -d`
Start with dashboard + worker	`docker compose --profile dashboard --profile worker up -d`
Start SonarQube	`docker compose -f docker-compose.sonar.yml up -d`
Connect API to SonarQube network (after every restart)	`docker network connect patchcourt-sonar_default patchcourt-api-1`
Check health	`curl http://localhost:8000/health`
Run demo review	`curl -X POST http://localhost:8000/api/review/demo`
Rebuild single service	`docker compose build api && docker compose up -d api`
View API logs	`docker logs patchcourt-api-1 --tail 50`

5.3 Network Architecture

Network	Members	Purpose
`patchcourt_default`	api, dashboard, postgres, redis, qdrant, worker	Internal service communication
`patchcourt-sonar_default`	sonarqube, api (manually joined)	SonarQube baseline access from API container

■ *The sonar-scanner-cli sibling container cannot reach SonarQube via localhost. _scanner_host_url() rewrites the URL to host.docker.internal so the scanner container reaches the host-mapped port 9000.*

5.4 Storage Volumes

Volume / Path	Type	Contents	Size
`pgdata`	Named volume	PostgreSQL audit log	~50 MB
`redisdata`	Named volume	Celery task queue, transient job state	~10 MB
`qdrantdata`	Named volume	RAG vector embeddings of policy documents	~20 MB
`sonardata/logs/extensions`	Named volumes	SonarQube embedded H2 DB, scan results, plugins	500MB-1GB
`./reports → /app/reports`	Host bind-mount	JSON + Markdown baseline reports, benchmark results	~1 MB/scan
`./.sandbox → container path`	Host bind-mount	Temp scan workdirs for sonar-scanner-cli. Deleted after scan.	Transient

6. API REFERENCE

All endpoints are served by FastAPI on port 8000. CORS is open (*). Errors return {"detail": {"category": "...", "message": "..."}} with HTTP 4xx/502.

Method + Path	Body / Params	Returns	Notes
`POST /api/review`	`{pr_url}`	ReviewReport (full)	Full pipeline: ingest→rag→evidence→agents →debate→judge. Saves audit to PG.
`POST /api/review/demo`	`(none)`	ReviewReport (demo)	Offline demo with synthetic PR. Forces mock LLM. Restores provider after.
`GET /api/baseline/latest`	`(none)`	BaselineReport JSON	Returns newest ./reports/baseline-*.json by mtime. No live SonarQube needed.
`POST /api/baseline`	`{pr_url}`	BaselineReport + patchcourt comparison	Fetches PR, runs on-demand SonarQube scan, runs full PatchCourt review, reconciles.
`POST /api/baseline/diff`	`{pr_url}`	BaselineReport (diff-only)	Scans only changed files/lines. Faster. Includes taint inference.
`GET /api/comparison/demo`	`(none)`	Demo comparison payload	Returns pre-built demo comparison data for the comparison tab.
`POST /api/benchmark/run`	`(none)`	Benchmark result	Runs labeled finding-adjudication benchmark and saves result.
`GET /api/benchmark/latest`	`(none)`	Latest benchmark JSON	Returns newest saved benchmark result from ./reports/benchmark-latest.json.
`GET /api/sonar/health`	`(none)`	{available, status}	Checks if SonarQube is reachable at SONAR_URL.
`GET /api/sonar/quality-gate`	`?component=`	Quality gate status	Uses configured project by default; pass component= for on-demand PR project.
`GET /api/sonar/measures`	`?component=`	Component metrics	Coverage, duplications, code smells, bugs, vulnerabilities.
`GET /api/sonar/issues`	`?page=&page;_size=&component;=`	Paginated issues	Paginated issue search. Pass component= for PR-specific project.
`GET /api/settings/llm`	`(none)`	LLMSettingsResponse	Current effective LLM settings (runtime overrides env).
`POST /api/settings/llm`	`{provider, model, api_key, base_url, use_mock_llm}`	LLMSettingsResponse	Update LLM settings at runtime. Re-initializes LLM client. No restart needed.
`POST /api/settings/llm/test`	`(none)`	{ok, detail}	Tests current LLM provider connection with a minimal generate call.
`POST /api/webhook/github`	`GitHub webhook payload`	202 Accepted	Enqueues review_pr Celery task. Returns 503 if REDIS_URL unset.
`GET /health`	`(none)`	{status: ok}	Liveness probe.

6.1 Error Category Reference

Category	HTTP	Cause
`llm_quota_exhausted`	502	Gemini/OpenAI 429 — daily/rate limit hit. Switch to mock or new key.
`llm_invalid_key`	502	401 from LLM provider. Check key in Settings.

Category	HTTP	Cause
`llm_service_busy`	502	503 from LLM provider. Retry in a moment.
`pr_not_found`	400	GitHub 404 — PR URL invalid or repo private without token.
`github_unreachable`	502	DNS/connection failure to api.github.com.
`sonar_offline`	502	Cannot reach SonarQube. Check container and network connect.
`timeout`	502	Request timed out (30s for GitHub, 600s for scanner).

7. CONFIGURATION (.env)

Copy .env.example to .env. All variables are optional — the system runs fully offline with no keys set (mock LLM + synthetic scanner).

Variable	Default	Purpose
`GITHUB_TOKEN`	(anonymous)	GitHub API token. Without it: 60 req/hr rate limit, no private repos.
`LLM_PROVIDER`	mock	mock | openai | claude | gemini | ollama
`LLM_API_KEY`	—	API key for openai / claude / gemini providers.
`LLM_BASE_URL`	https://api.openai.com/v1	Override endpoint for openai-compatible servers (vLLM, LM Studio, Ollama).
`LLM_MODEL`	gpt-4o	Model name passed to the provider.
`USE_MOCK_LLM`	auto	Force deterministic mock regardless of provider setting.
`PATCHCOURT_TOOLS`	semgrep,bandit,gitleaks,radon	Comma-separated list of enabled static-analysis adapters.
`USE_SYNTHETIC_TOOLS`	false	Add offline synthetic scanner (always produces demo findings).
`SANDBOX_ENABLED`	false	Run tools inside Docker sandbox container (--network none).
`SANDBOX_IMAGE`	patchcourt-sandbox:latest	Docker image for the sandbox runner.
`SANDBOX_WORKDIR`	(system tmp)	Host path for per-scan workdirs. Must be visible to Docker daemon.
`QDRANT_URL`	(in-memory)	Qdrant server URL. Empty = in-memory store (no persistence).
`QDRANT_COLLECTION`	patchcourt_chunks	Qdrant collection name for RAG chunks.
`REDIS_URL`	(no async)	Celery broker URL. Empty = webhook returns 503.
`DATABASE_URL`	(no audit)	SQLAlchemy URL for audit trail. Empty = no persistence.
`SONAR_URL`	http://localhost:9000	SonarQube server URL. Inside Docker: http://patchcourt-sonar-sonarqube-1:9000
`SONAR_TOKEN`	—	SonarQube user token. Required for baseline scan and issue fetch.
`SONAR_COMPONENT`	patchcourt	Default SonarQube project key for the live dashboard panels.

7.1 LLM Provider Quick Reference

Provider	LLM_PROVIDER	Needs	Default Model
Mock (offline)	`mock`	Nothing	— (deterministic stub)
OpenAI	`openai`	LLM_API_KEY	`gpt-4o`
Anthropic Claude	`claude`	LLM_API_KEY	`claude-3-5-sonnet-latest`
Google Gemini (free tier)	`gemini`	LLM_API_KEY (AI Studio)	`gemini-flash-latest`
Local Ollama	`ollama`	Running Ollama server	`llama3.1`

■ *LLM settings can be changed at runtime via POST /api/settings/llm — no container restart needed. The dashboard Settings panel uses this endpoint.*

8. SONARQUBE BASELINE

The SonarQube baseline compares unresolved static-analysis issues against the files a GitHub PR actually changes. It runs an on-demand scan of the PR's changed files into a per-PR SonarQube project, then reconciles the results with PatchCourt's adjudication.

8.1 On-Demand Scan Flow

`1. fetch_pr()`	Fetches PR metadata, diff, raw file contents, and base file contents from GitHub REST API.
`2. project_key_for_pr()`	Generates a unique SonarQube project key: pr-{owner}-{repo}-{num} (lowercased, sanitized).
`3. run_ondemand_scan()`	Writes changed files to a temp workdir, runs sonarsource/sonar-scanner-cli:11 as a sibling container, waits for CE processing via _wait_for_ce().
`4. build_report()`	Fetches issues from SonarQube API, cross-references with PR changed files, marks each issue in_pr=True/False.
`5. run_review()`	Runs full PatchCourt review on the same PR (agents + judge).
`6. compare_findings()`	Aligns SonarQube issues to FindingAssessments by file:line (fallback: file+rule). Groups into confirmed/contradicted/uncertain/unmatched.
`7. write_report()`	Saves baseline-{owner}-{repo}-{num}-{ts}.json and .md to ./reports/.

8.2 Comparison Groups

Group	Border Colour	Meaning
Confirmed / kept	Green	SonarQube issue backed by a corroborated/tool PatchCourt claim on same file:line.
Down-tiered / unbacked	Orange	SonarQube issue with no PatchCourt support — likely false positive.
Sonar-missed	Primary (teal)	Strong PatchCourt tool claim that SonarQube never reported.
Contradicted	Red	PatchCourt adjudication contradicts the SonarQube finding.
Uncertain	Yellow	Insufficient evidence; requires human review.
Unmatched	Zinc	No PatchCourt assessment matched this SonarQube issue.

8.3 Key Implementation Notes

- All SonarQube findings carry provenance_tier=1 (T1) regardless of severity. Severity is SonarQube's own judgement; tier describes evidence provenance.
- The scanner image runs as uid 1000. The workdir is chmod 755 so it can read the bind-mounted files.
- _wait_for_ce() polls /api/ce/component every 5s up to 300s to wait for SonarQube's async Compute Engine processing.
- Per-PR project key format: pr-{owner}-{repo}-{num} — never collides with the main patchcourt project.
- GET /api/baseline/latest reads the newest .json by mtime — no database index needed.
- The dashboard Baseline tab loads the latest report on mount (no SonarQube needed). Refresh baseline button calls POST /api/baseline (needs SonarQube running, ~2-3 min).

9. DASHBOARD (Next.js 15)

The dashboard runs on port 3001 (mapped 3001:3000). NEXT_PUBLIC_API_BASE defaults to http://localhost:8000. Built with shadcn/ui components, Tailwind CSS, and Lucide icons.

9.1 Tab Structure

Tab / Route	Component	What It Shows
`Overview /`	`app/page.tsx`	KPI cards (score, verdict, file count), verdict banner, submit PR URL, Offline Demo button.
`Evidence /evidence`	`app/evidence/page.tsx`	Full evidence docket — all claims with tier badges, severity, corroboration pills, agent labels. Filter/sort by tier, severity, agent.
`Debate /debate`	`app/debate/page.tsx`	Debate transcripts — supporting/opposing arguments per conflict, round count, verdict summary, code snippet.
`Baseline /baseline`	`app/baseline/page.tsx+ sonar-baseline-panel.t sx`	SonarQube baseline: severity breakdown, PatchCourt verdict, confirmed/down-tiered/sonar-missed sections with border-left cards and code snippets.
`Comparison /comparison`	`app/comparison/page.ts x`	Finding-by-finding adjudication: SonarQube issue vs PatchCourt assessment. Fault-risk badge, QA badge, FP/FN root cause block, taint inference section.

9.2 Key Frontend Files

`lib/types.ts`	All TypeScript types: Report, FindingAssessmentView, BaselineReport, FindingComparison, etc.
`lib/api.ts`	API client: submitReview, runDemo, runComparisonDemo, refreshBaseline, getSonarIssues, etc.
`lib/review-context.tsx`	React context: report, phase, setReport, setPhase, setError. Shared across all tabs.
`components/sonar-baseline-pan el.tsx`	Baseline tab panel. Loads latest on mount. Refresh button calls POST /api/baseline.
`components/layout-wrapper.tsx`	Sidebar nav + tab routing wrapper.
`app/comparison/page.tsx`	Full comparison page: CodeSnippet, border-left finding cards, Load Demo button.

9.3 Comparison Tab — Special Features

- CodeSnippet component: renders numbered lines around the flagged line. Arrow (→) marks the target line with yellow highlight background.
- Load Demo button: calls GET /api/comparison/demo fresh every click, wraps payload into Report shape, calls setReport(). Avoids stale baseline_report.
- Fault-risk badge: colour-coded border badge (red=high, yellow=medium, zinc=low) from fault_correlation field on FindingAssessment.
- QA badge: shown only when qa_verdict != VALID (INCONSISTENT=red, NEEDS_REWORK=yellow).
- FP/FN root cause block: orange left-border block showing root cause category and explanation.
- Taint inference section: purple border cards for tool='taint-inference' assessments. Labelled 'LLM-Inferred Taint Flows — Scanner Missed'.
- NEW IN PR badge: shown when introduced_by_pr=True on the FindingAssessment.

10. DATA MODELS (agents/schemas.py)

10.1 Core Pydantic Models

Model	Key Fields	Used By
`Evidence`	tier: int (1-5), text: str	Attached to every Claim and FindingAssessment
`Claim`	agent, issue, file, line, severity(1-5), evidence[], confidence, tier, corroborated, source('tool'|'llm')	Output of all three agents; input to judge
`DebatedClaim`	claim: Claim, supporting, opposing, rounds, code_snippet, verdict_summary	Output of debate stage; QA-Checker validates this
`FindingAssessment`	tool, file, line, rule, message, severity, category, status, rationale, corroborated, corroborating_tools, policies[], contextual_priority, contextual_action, evidence[], fp_fn_root_cause, fp_fn_explanation, fault_correlation, qa_verdict, qa_note, code_snippet, introduced_by_pr, pre_existing	Output of FindingValidator; displayed in baseline and comparison tabs
`ReviewReport`	pr_url, overall_score, verdict, files[], claims[], debate_transcripts[], finding_assessments[], correlations[], contextual_summary, baseline_report	Final output of run_review(); serialized to JSON by API
`PRContext`	owner, repo, pr_number, title, body, diff, changed_files[], file_contents{}, base_file_contents{}, patches{}, rag_query	Passed through entire pipeline; populated by ingest node
`CorrelationGroup`	file, line, tools[], findings[], relationship, severity_range, disputed	Output of correlate_findings(); used by FindingValidator

10.2 FindingAssessment — Extended Fields (Research Features)

Field	Values	Set By
`fp_fn_root_cause`	Flawed Rule Spec | Inconsistent Rule Implementation | Unhandled Language Features | Missing Cases | Mishandling IR | Analysis Module Error | General Programming Error	_fp_fn_root_cause() in validator.py
`fault_correlation`	high | medium | low	_fault_correlation() in validator.py
`qa_verdict`	VALID | INCONSISTENT | NEEDS_REWORK	run_qa_checks() in qa_checker.py
`introduced_by_pr`	True if flagged line is new in this PR	_classify_finding() in validator.py (needs base_file_contents)
`code_snippet`	5-line context window with → marker on flagged line	_code_snippet() in validator.py

11. TROUBLESHOOTING

502 Bad Gateway on POST /api/baseline

Cause: Root cause: fetch_pr() hits GitHub rate limit (60 req/hr unauthenticated), or run_review() inside baseline_compare fails (LLM quota, network).

Fix: Set GITHUB_TOKEN in .env. Check docker logs patchcourt-api-1 for the actual exception category.

SonarQube baseline shows 0 issues

Cause: The container's /app/reports has an old report from a different PR/component, or GET /api/baseline/latest returned a report before the on-demand scan completed.

Fix: Click Refresh baseline with the correct PR URL. Wait ~2-3 min for scanner to finish.

Score shows inflated value (e.g. 1562%)

Cause: _fault_multiplier was compounding with corroboration_factor (1.75x * 1.3x = ~2.3x). Fixed: fault_multiplier is now ±15% nudge only on T1/T2 tool claims.

Fix: Verify scoring.py uses _FAULT_CORR_NUDGE = {high:1.15, medium:1.0, low:0.85}.

Comparison tab shows stale data

Cause: report.baseline_report is set once when runDemo() runs POST /api/review/demo. Navigating to comparison tab just reads that stale value.

Fix: Use the Load Demo button — it calls GET /api/comparison/demo fresh every click.

patchcourt-sonar-sonarqube-1 not reachable from API

Cause: The two compose stacks (patchcourt and patchcourt-sonar) are on separate networks. The API container is not automatically joined to patchcourt-sonar_default.

Fix: Run: docker network connect patchcourt-sonar_default patchcourt-api-1 (after every restart)

sonar-scanner-cli cannot reach SonarQube

Cause: The scanner runs as a sibling container and cannot use localhost or the container hostname.

Fix: _scanner_host_url() rewrites the URL to host.docker.internal automatically. Verify SONAR_URL in .env does not use a hostname only resolvable inside the patchcourt network.

Dashboard shows blank / no data after restart

Cause: The dashboard container was rebuilt but the API container still has old code, or the reports volume is empty.

Fix: docker compose build api && docker compose up -d api. Then click Load Demo or Refresh baseline.

LLM quota exhausted (Gemini free tier)

Cause: Gemini free tier has daily and per-minute rate limits.

Fix: Switch to mock mode via Settings panel (POST /api/settings/llm {use_mock_llm: true}). Or use a different API key / provider.

Celery worker not processing webhooks

Cause: REDIS_URL not set, or worker container not started.

Fix: docker compose --profile worker up -d. Check docker logs patchcourt-worker-1.

Sandbox scan produces no findings

Cause: SANDBOX_ENABLED=false (default), or sandbox image not built.
Fix: docker compose build sandbox. Set SANDBOX_ENABLED=1 in .env.

12. TESTS

Run: pytest -q (from repo root). 51+ tests. asyncio_mode=auto.

Test File	Covers
`test_judge.py`	Tier weights, scoring math, BLOCK-on-T5-alone rule, safe_verdict(), fault_multiplier
`test_validator.py`	FindingValidator: SUPPORTED/CONTRADICTED/UNCERTAIN/OUT_OF_SCOPE, fp_fn_root_cause, fault_correlation
`test_evidence.py`	EvidenceEngine, normalize_findings(), corroborate_claims(), tier assignment
`test_conflict.py`	detect_conflicts() — conflict gating logic
`test_debate_context.py`	Debate context building, retrieval-gated rebuttal
`test_baseline.py`	SonarQube baseline: compare_findings(), compare_with_review(), markdown()
`test_baseline_api.py`	POST /api/baseline endpoint integration
`test_comparison.py`	build_review_comparison(), finding comparison groups
`test_pipeline.py`	Offline end-to-end: real LangGraph + RAG + judge with fake LLM
`test_ingest.py`	parse_pr_url(), PR URL validation
`test_sandbox.py`	Sandbox graceful-skip when Docker unavailable
`test_audit.py`	PostgreSQL audit trail: store_audit(), model schema
`test_llm_providers.py`	LLM provider dispatch: mock/openai/claude/gemini/ollama
`test_llm_settings_api.py`	GET/POST /api/settings/llm runtime config endpoints
`test_benchmark.py / test_benchmark_api.py`	Adjudication benchmark runner and API endpoint
`test_contextual.py`	assess_context(), contextual_priority, contextual_action mapping
`test_correlation.py`	correlate_findings(), correlation_index(), corroborating_tools()
`test_rules.py`	rule_for(), category_for(), match_policies(), load_repository_policies()
`test_tiers.py`	TIER_WEIGHTS values, tier boundary conditions

13. QUICK-REFERENCE CHEATSHEET

13.1 Full Startup Sequence (Clean State)

- 1. git clone && cd patchcourt
- 2. cp .env.example .env # add GITHUB_TOKEN, LLM_API_KEY, SONAR_TOKEN
- 3. docker compose build sandbox
- 4. docker compose -f docker-compose.sonar.yml up -d
- 5. docker compose --profile dashboard --profile worker up -d
- 6. docker network connect patchcourt-sonar_default patchcourt-api-1
- 7. curl http://localhost:8000/health # → {status: ok}
- 8. curl http://localhost:8000/api/sonar/health # → {available: true}
- 9. Open http://localhost:3001 in browser
- 10. Click Load Demo or paste a GitHub PR URL

13.2 Git Remotes

Remote	URL	Purpose
`mine`	github.com/Baalaa06/patchcourt1	Personal fork — push feature branches here
`origin`	github.com/sakthiMurugan03/patchcourt	Team upstream — PRs and demo PRs here

13.3 Key Score Thresholds

Threshold	Value	Effect
`block_threshold`	10.0	overall_score >= 10.0 → BLOCK (if non_t5_score also >= 10.0)
`needs_review_threshold`	4.0	overall_score >= 4.0 → NEEDS_REVIEW
`debate_max_rounds`	2	Maximum debate rounds per conflict
`taint confidence min`	0.4	Minimum LLM confidence to emit a taint FindingAssessment
`taint flow max distance`	50 lines	Source and sink must be within 50 lines (same-function heuristic)

13.4 Useful One-Liners

Action	Command
Offline demo (no Docker)	`python -m patchcourt review --demo`
Review a real PR	`python -m patchcourt review https://github.com/owner/repo/pull/N`
Run all tests	`pytest -q`
Check what's in latest baseline	`curl -s http://localhost:8000/api/baseline/latest | python -m json.tool | head -40`
Force mock LLM at runtime	`curl -X POST http://localhost:8000/api/settings/llm -H "Content-Type: application/json" -d '{"use_mock_llm":true}'`
Check SonarQube issues for PR project	`curl http://localhost:8000/api/sonar/issues?component=pr-owner-repo-N`
Tail API logs	`docker logs patchcourt-api-1 -f --tail 30`
Restart just the API	`docker compose up -d api`

Action	Command
Rebuild dashboard only	`docker compose build dashboard && docker compose up -d dashboard`
List all baseline reports	`dir reports\baseline-*.json /b /o-d`

13.5 Policies Directory (patchcourt/policies/)

`security-policy.md`	Hardcoded secrets forbidden. All security findings adjudicated before merge.
`coding-standards.md`	Style, naming, complexity, dead-code rules.
`api-boundaries.md`	Internal members stay private; visibility widening requires justification.
`repository-workflow.md`	PR process, review requirements, merge criteria.